

Geometric Methods in Computer Vision (EE60030)

Course Project: A Comparative Review of Neural Radiance Fields (NeRF)
and 3D Gaussian Splatting for Novel View Synthesis

Rickarya Das – 23EE30019
Department of Electrical Engineering

April 6, 2026

Abstract

This report presents a mathematically rigorous comparative review of two seminal works in neural scene representation and novel view synthesis: **Neural Radiance Fields (NeRF)** [1] and **3D Gaussian Splatting (3DGS)** [2]. Both methods address the problem of synthesizing photorealistic novel views of a scene from a sparse set of calibrated input images—a core challenge in geometric computer vision. NeRF represents scenes as continuous 5D implicit functions optimized via differentiable volume rendering, while 3DGS introduces an explicit, anisotropic 3D Gaussian primitive representation with a tile-based differentiable rasterizer achieving real-time rendering. We provide a thorough derivation of the mathematical foundations underpinning each method—including volume rendering theory, positional encoding, hierarchical sampling, Gaussian splatting, covariance parameterization, and spherical harmonics—and critically compare their geometric assumptions, optimization strategies, rendering pipelines, and empirical performance. Our analysis situates both methods within the broader context of projective geometry, differentiable rendering, and 3D reconstruction covered in this course.

Contents

1	Introduction and Motivation	3
2	Mathematical Preliminaries	3
2.1	Camera Model and Ray Casting	3
2.2	Volume Rendering Equation	4
2.3	Numerical Quadrature Approximation	5

2.4	Spherical Harmonics for View-Dependent Color	5
3	Neural Radiance Fields (NeRF)	7
3.1	Scene Representation	7
3.2	Positional Encoding	8
3.3	Hierarchical Volume Sampling	8
3.4	Training Objective	9
3.5	NeRF Pipeline Summary	10
4	3D Gaussian Splatting (3DGS)	10
4.1	Scene Representation: Anisotropic 3D Gaussians	10
4.2	Covariance Parameterization	11
4.3	Projection to 2D: EWA Splatting	11
4.4	Alpha-Compositing Rendering	12
4.5	Tile-Based Differentiable Rasterizer	12
4.6	Optimization with Adaptive Density Control	13
4.6.1	Loss Function	13
4.6.2	Adaptive Gaussian Densification	13
4.7	3DGS Pipeline Summary	15
5	Comparative Analysis: NeRF vs. 3D Gaussian Splatting	16
5.1	Representation	17
5.2	Rendering Pipeline	17
5.3	Quantitative Performance	17
5.4	Geometric Analysis	17
5.5	Implicit Smoothing vs. Explicit Detail	18
6	Connection to Course Topics in GMCV	18
6.1	Projective Geometry and Camera Models	18
6.2	Epipolar Geometry and Multi-View Consistency	19
6.3	Structure from Motion (SfM)	19
6.4	Optimization on Manifolds	19
7	Illustrative Code Implementations	19
7.1	NeRF: Positional Encoding and Volume Rendering	19
7.2	3DGS: Gaussian Projection and Covariance Computation	20
8	Discussion and Conclusion	21
8.1	Summary of Findings	21
8.2	Limitations and Open Problems	21
8.3	Future Directions	21

1 Introduction and Motivation

Novel view synthesis (NVS) is the task of generating photorealistic images of a scene from arbitrary camera viewpoints, given a set of calibrated input images. This problem lies at the intersection of several topics covered in this course: camera models and calibration, epipolar geometry, projective transformations, and 3D reconstruction.

Classical approaches to NVS relied on explicit geometric reconstruction via Structure-from-Motion (SfM) and Multi-View Stereo (MVS), followed by image-based rendering. While effective, these methods struggle with complex view-dependent effects (specularities, translucency) and incomplete reconstructions.

Two recent paradigms have revolutionized this field:

1. **Neural Radiance Fields (NeRF)** [1]: Represents scenes as *implicit* continuous volumetric functions parameterized by neural networks, optimized via differentiable volume rendering.
2. **3D Gaussian Splatting (3DGS)** [2]: Represents scenes as *explicit* collections of anisotropic 3D Gaussian primitives, rendered via a differentiable tile-based rasterizer.

Both methods share the same image formation model rooted in classical volume rendering, yet differ fundamentally in scene representation (implicit vs. explicit) and rendering strategy (ray marching vs. rasterization). This report provides a unified mathematical treatment of both approaches and a critical comparative analysis.

2 Mathematical Preliminaries

2.1 Camera Model and Ray Casting

A calibrated pinhole camera with intrinsic matrix $\mathbf{K} \in \mathbb{R}^{3 \times 3}$ and extrinsic parameters $[\mathbf{R} \mid \mathbf{t}]$ maps a 3D world point $\mathbf{X} \in \mathbb{R}^3$ to pixel coordinates $\mathbf{u} = (u, v)^T$ via:

$$\lambda \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \mathbf{K}[\mathbf{R} \mid \mathbf{t}] \begin{pmatrix} \mathbf{X} \\ 1 \end{pmatrix} \quad (1)$$

For rendering, we invert this process. Given a pixel (u, v) , we cast a ray from the camera center $\mathbf{o} \in \mathbb{R}^3$ through the pixel into the scene:

$$\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}, \quad t \in [t_n, t_f] \quad (2)$$

where $\mathbf{d} = \mathbf{R}^T \mathbf{K}^{-1}(u, v, 1)^T / \|\mathbf{K}^{-1}(u, v, 1)^T\|$ is the normalized ray direction, and $[t_n, t_f]$ are the near and far bounds.

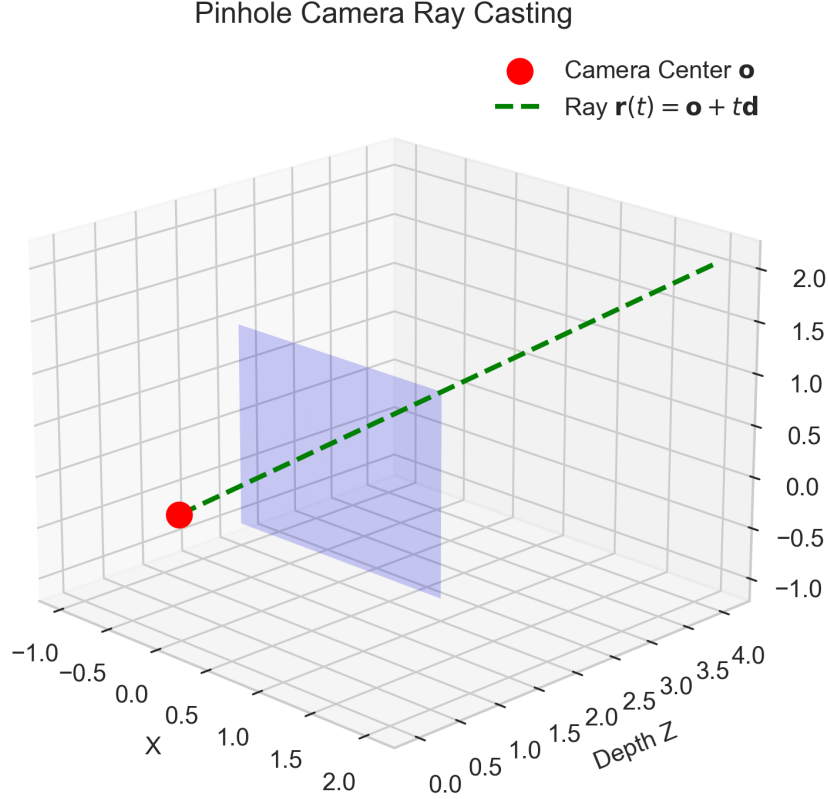


Figure 1: Pinhole camera model illustrating the rendering ray cast from the camera center into the 3D scene.

2.2 Volume Rendering Equation

The expected color $C(\mathbf{r})$ along ray $\mathbf{r}$ through a participating medium with spatially varying density $\sigma(\mathbf{x})$ and color $\mathbf{c}(\mathbf{x}, \mathbf{d})$ is given by the volume rendering integral:

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t) \sigma(\mathbf{r}(t)) \mathbf{c}(\mathbf{r}(t), \mathbf{d}) dt \quad (3)$$

where the **accumulated transmittance** $T(t)$ denotes the probability that the ray travels from t_n to t without being absorbed:

$$T(t) = \exp\left(-\int_{t_n}^t \sigma(\mathbf{r}(s)) ds\right) \quad (4)$$

Geometric interpretation: The quantity $\sigma(\mathbf{r}(t)) dt$ represents the differential probability of the ray being stopped in $[t, t+dt]$. The term $T(t)\sigma(\mathbf{r}(t))$ is therefore the probability density that the ray terminates exactly at t , and the integral in Eq. (3) computes the expected color under this distribution.

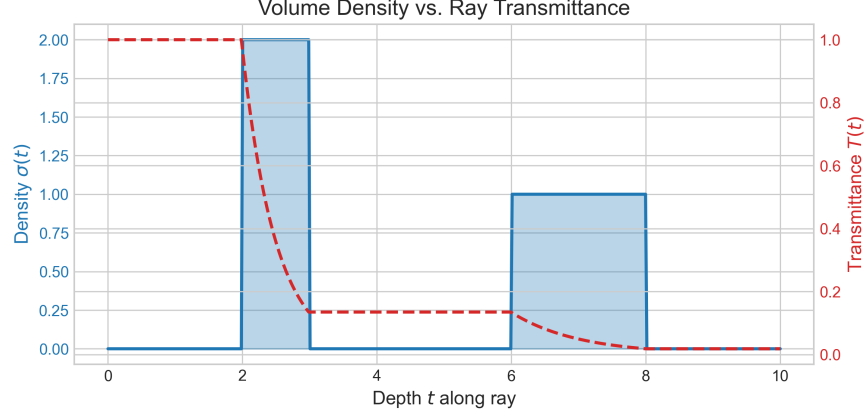


Figure 2: Visualization of volume density $\sigma(t)$ and the corresponding monotonically decreasing transmittance $T(t)$ along a ray.

2.3 Numerical Quadrature Approximation

In practice, we approximate Eq. (3) using N discrete samples $\{t_i\}_{i=1}^N$ along the ray:

$$\hat{C}(\mathbf{r}) = \sum_{i=1}^N T_i \alpha_i \mathbf{c}_i \quad (5)$$

where:

$$\alpha_i = 1 - \exp(-\sigma_i \delta_i), \quad T_i = \prod_{j=1}^{i-1} (1 - \alpha_j), \quad \delta_i = t_{i+1} - t_i \quad (6)$$

Here $\alpha_i \in [0, 1]$ is the opacity of the i -th sample and T_i is the discrete transmittance. This formulation is *differentiable* with respect to σ_i and $\mathbf{c}_i$, enabling gradient-based optimization.

2.4 Spherical Harmonics for View-Dependent Color

Both NeRF and 3DGS model view-dependent appearance. Spherical harmonics (SH) provide an efficient basis for representing functions on the sphere $\mathbb{S}^2$. The real SH basis functions of degree l and order m are:

$$Y_l^m(\theta, \phi) = \begin{cases} \sqrt{2} K_l^m \cos(m\phi) P_l^m(\cos \theta) & m > 0 \\ K_l^0 P_l^0(\cos \theta) & m = 0 \\ \sqrt{2} K_l^{|m|} \sin(|m|\phi) P_l^{|m|}(\cos \theta) & m < 0 \end{cases} \quad (7)$$

where P_l^m are the associated Legendre polynomials and K_l^m is a normalization constant. A view-dependent color function is approximated as:

$$c(\mathbf{d}) = \sum_{l=0}^{l_{\max}} \sum_{m=-l}^l k_l^m Y_l^m(\mathbf{d}) \quad (8)$$

with optimizable coefficients k_l^m per color channel. Using $l_{\max} = 3$ yields $(l_{\max} + 1)^2 = 16$ coefficients per channel.



Figure 3: 3D polar plots demonstrating selected Spherical Harmonics (Y_2^0 and Y_2^1) basis functions used for representing complex view-dependent photometric variations.

3 Neural Radiance Fields (NeRF)

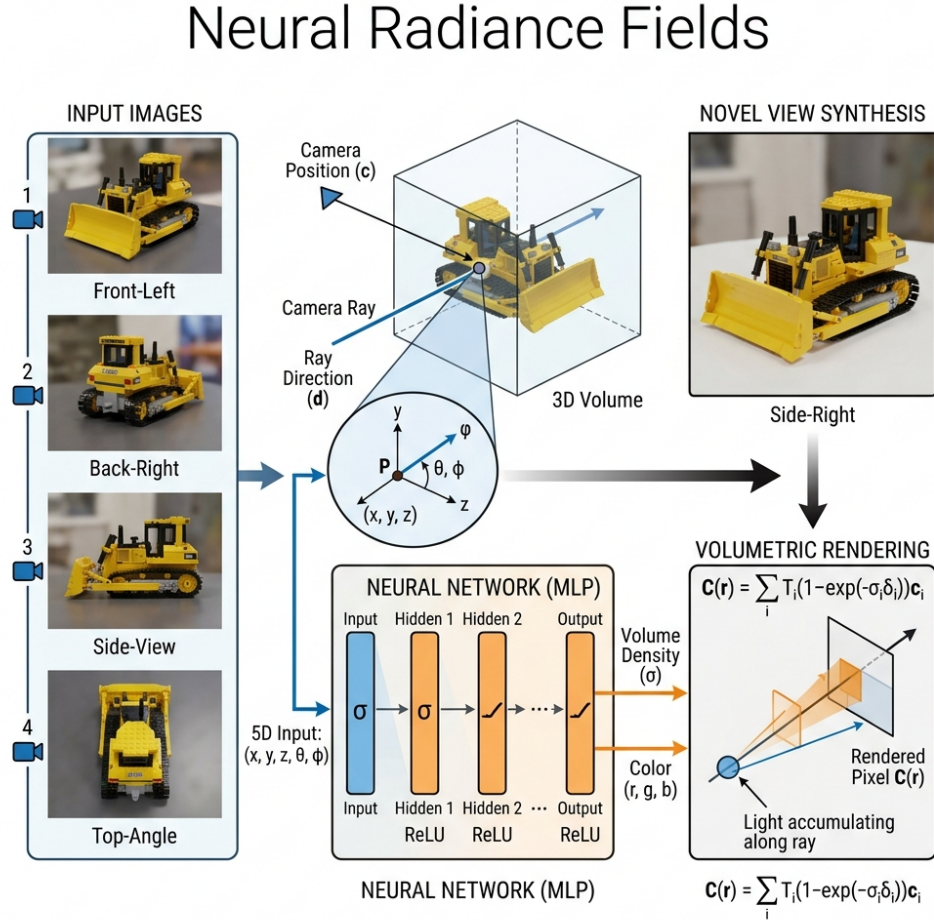


Figure 4: Overview of the NeRF pipeline: Input images as 5D vectors are passed through a Multi-Layer Perceptron (MLP) to output volume density and view-dependent color, followed by volume rendering to synthesize novel views.

3.1 Scene Representation

NeRF represents a static scene as a continuous function F_{Θ} parameterized by a Multi-Layer Perceptron (MLP) with weights Θ :

$$F_{\Theta} : (\mathbf{x}, \mathbf{d}) \mapsto (\mathbf{c}, \sigma), \quad \mathbf{x} \in \mathbb{R}^3, \mathbf{d} \in \mathbb{S}^2, \mathbf{c} \in [0, 1]^3, \sigma \in \mathbb{R}_{\geq 0} \quad (9)$$

A critical design choice enforces **multi-view consistency**: the density σ depends only on position $\mathbf{x}$ (not viewing direction), while color $\mathbf{c}$ depends on both $\mathbf{x}$ and $\mathbf{d}$. This is implemented architecturally by processing $\mathbf{x}$ through 8 fully-connected layers (256 units each, ReLU activations) to produce σ and a feature vector, which is then concatenated with $\mathbf{d}$ and passed through an additional layer to produce $\mathbf{c}$.

3.2 Positional Encoding

MLPs are biased towards learning low-frequency functions—a phenomenon formalized by the *spectral bias* or *Neural Tangent Kernel* (NTK) theory [7]. To represent high-frequency geometric and photometric details, NeRF lifts inputs to a higher-dimensional space via a positional encoding $\gamma : \mathbb{R} \rightarrow \mathbb{R}^{2L}$:

$$\gamma(p) = \left(\sin(2^0 \pi p), \cos(2^0 \pi p), \sin(2^1 \pi p), \cos(2^1 \pi p), \dots, \sin(2^{L-1} \pi p), \cos(2^{L-1} \pi p) \right) \quad (10)$$

This is applied independently to each coordinate. For spatial coordinates, $L = 10$ (mapping $\mathbb{R}^3 \rightarrow \mathbb{R}^{60}$); for viewing directions, $L = 4$ (mapping $\mathbb{R}^3 \rightarrow \mathbb{R}^{24}$). The lower frequency encoding for $\mathbf{d}$ reflects the physical prior that view-dependent effects are smoother than geometric detail.

Theoretical justification: The positional encoding can be interpreted as a deterministic approximation to the Random Fourier Features kernel [8]. The encoded MLP $F_\Theta \circ \gamma$ effectively corresponds to a kernel regression with a tunable bandwidth, enabling representation of arbitrarily high-frequency signals.

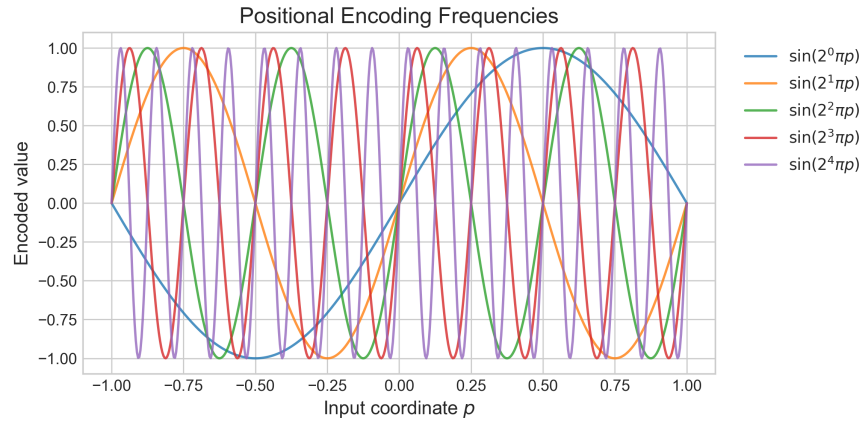


Figure 5: Positional encoding maps low-dimensional inputs into a higher-dimensional frequency space using sinusoidal wave expansions, overcoming the inherent spectral bias of MLPs.

3.3 Hierarchical Volume Sampling

Naïvely sampling points uniformly along each ray is inefficient: most samples fall in free space or occluded regions. NeRF addresses this with a two-network hierarchical strategy.

Coarse network: N_c samples are drawn via stratified sampling (one sample per bin from a uniform partition of $[t_n, t_f]$):

$$t_i \sim \mathcal{U} \left[t_n + \frac{i-1}{N_c} (t_f - t_n), t_n + \frac{i}{N_c} (t_f - t_n) \right], \quad i = 1, \dots, N_c \quad (11)$$

Fine network: The coarse network’s output defines a piecewise-constant PDF along the ray. Normalizing the weights $w_i = T_i \alpha_i$ from Eq. (5):

$$\hat{w}_i = \frac{w_i}{\sum_{j=1}^{N_c} w_j} \quad (12)$$

An additional N_f samples are drawn from this distribution via inverse transform sampling. The fine network is evaluated at all $N_c + N_f$ samples to produce the final rendered color.

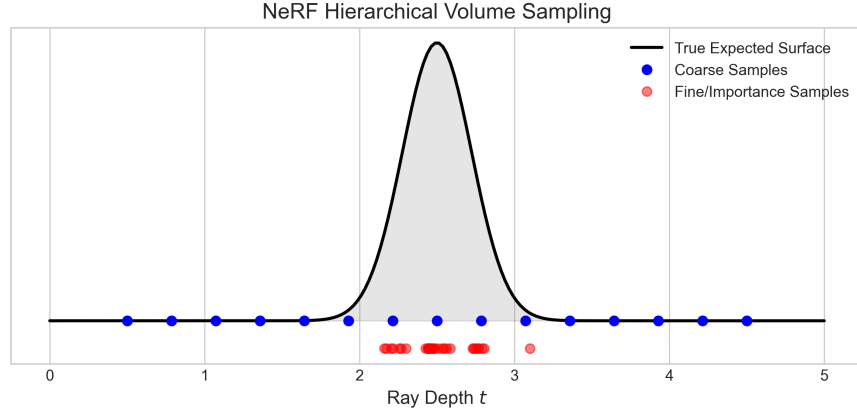


Figure 6: Hierarchical volume sampling concentrates the fine network points (red) tightly around the expected scene surface geometries discovered by the coarse uniform samples (blue).

3.4 Training Objective

The total loss is the sum of coarse and fine photometric reconstruction errors:

$$\mathcal{L} = \sum_{\mathbf{r} \in \mathcal{R}} \left[\left\| \hat{C}_c(\mathbf{r}) - C_{gt}(\mathbf{r}) \right\|_2^2 + \left\| \hat{C}_f(\mathbf{r}) - C_{gt}(\mathbf{r}) \right\|_2^2 \right] \quad (13)$$

where $\mathcal{R}$ is a batch of rays, $\hat{C}_c$ and $\hat{C}_f$ are the coarse and fine rendered colors, and C_{gt} is the ground-truth pixel color.

3.5 NeRF Pipeline Summary

Algorithm 1 NeRF Training (per iteration)

- 1: Sample a batch of rays $\mathcal{R}$ from training images
 - 2: **for** each ray $\mathbf{r} \in \mathcal{R}$ **do**
 - 3: Sample N_c stratified points along $\mathbf{r}$
 - 4: Evaluate coarse MLP: $(\mathbf{c}_i^c, \sigma_i^c) = F_{\Theta_c}(\gamma(\mathbf{x}_i), \gamma(\mathbf{d}))$
 - 5: Render coarse color $\hat{C}_c(\mathbf{r})$ via Eq. (5)
 - 6: Importance-sample N_f additional points from coarse weights
 - 7: Evaluate fine MLP at all $N_c + N_f$ points
 - 8: Render fine color $\hat{C}_f(\mathbf{r})$ via Eq. (5)
 - 9: **end for**
 - 10: Update Θ_c, Θ_f by minimizing $\mathcal{L}$ (Eq. (13)) via Adam
-

4 3D Gaussian Splatting (3DGS)

4.1 Scene Representation: Anisotropic 3D Gaussians

3DGS represents the scene as a collection of M anisotropic 3D Gaussian primitives $\{G_k\}_{k=1}^M$.



Figure 7: Visualization of 3D anisotropic Gaussian primitives. These ellipsoids can be stretched and rotated to capture complex geometry more efficiently than isotropic point clouds.

Each Gaussian is defined by the parameters $\{\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k, \alpha_k, \mathbf{k}_k^{SH}\}$:

- **Mean** $\boldsymbol{\mu}_k \in \mathbb{R}^3$: center position in world coordinates.
- **Covariance** $\boldsymbol{\Sigma}_k \in \mathbb{R}^{3 \times 3}$: positive semi-definite matrix defining the ellipsoidal shape.
- **Opacity** $\alpha_k \in [0, 1]$: learned per-Gaussian opacity (sigmoid-activated).
- **SH coefficients** $\mathbf{k}_k^{SH}$: spherical harmonic coefficients for view-dependent color.

The influence of the k -th Gaussian at a point $\mathbf{x}$ is:

$$G_k(\mathbf{x}) = \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^T \boldsymbol{\Sigma}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k)\right) \quad (14)$$

4.2 Covariance Parameterization

Directly optimizing $\boldsymbol{\Sigma}$ via gradient descent does not guarantee positive semi-definiteness. Instead, 3DGS parameterizes the covariance via a rotation matrix $\mathbf{R} \in SO(3)$ (stored as a unit quaternion $\mathbf{q} \in \mathbb{S}^3$) and a scaling vector $\mathbf{s} \in \mathbb{R}_{>0}^3$ (exponential activation):

$$\boldsymbol{\Sigma} = \mathbf{R} \mathbf{S} \mathbf{S}^T \mathbf{R}^T, \quad \mathbf{S} = \text{diag}(\mathbf{s}) \quad (15)$$

This decomposition is analogous to the eigendecomposition $\boldsymbol{\Sigma} = \mathbf{R} \boldsymbol{\Lambda} \mathbf{R}^T$ where $\boldsymbol{\Lambda} = \mathbf{S}^2$. The quaternion representation has four parameters but the unit-norm constraint leaves three degrees of freedom, matching $SO(3)$. Combined with three scaling parameters, this gives six parameters for the symmetric 3×3 covariance—exactly the correct number of degrees of freedom.

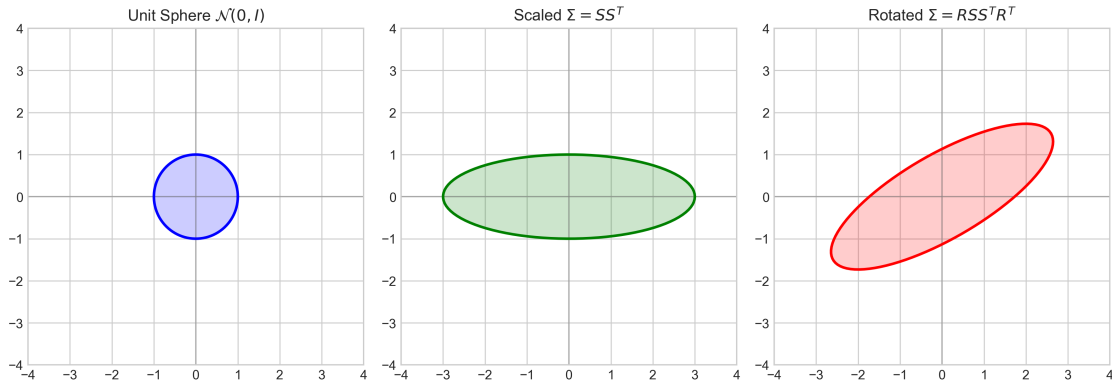


Figure 8: Geometric steps demonstrating matrix scale ($\mathbf{S}$) and rotation ($\mathbf{R}$) applied to a standard unit normal distribution, safely forming unconstrained positive semi-definite 3D Gaussian structure.

4.3 Projection to 2D: EWA Splatting

To render, each 3D Gaussian is projected to 2D image space using the **Elliptical Weighted Average (EWA)** splatting framework [3]. Given a viewing transformation $\mathbf{W}$ (world-to-

camera) and the Jacobian $\mathbf{J}$ of the local affine approximation to the perspective projection:

$$\Sigma' = \mathbf{J} \mathbf{W} \Sigma \mathbf{W}^T \mathbf{J}^T \quad (16)$$

The 2D Gaussian in screen space is obtained by extracting the top-left 2×2 submatrix of the 3×3 matrix Σ' . The Jacobian $\mathbf{J}$ accounts for the nonlinearity of the perspective division $(x/z, y/z)$ via a first-order Taylor expansion around the Gaussian mean.

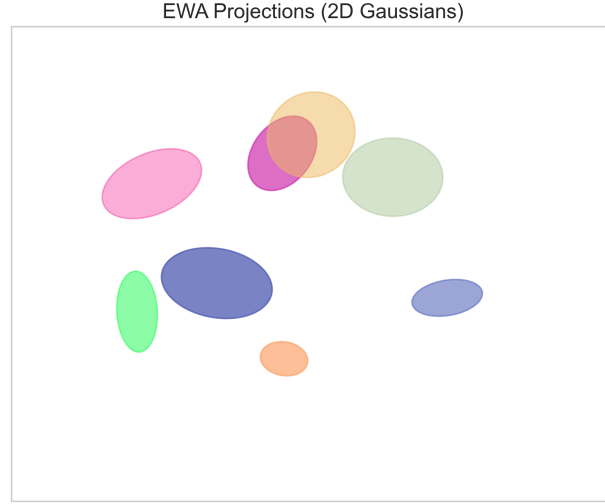


Figure 9: EWA splatting conceptually translates unstructured 3D Gaussian volumes down to analytically bounded mathematically accurate 2D ellipses projected directly upon the observer’s viewing plane.

4.4 Alpha-Compositing Rendering

The pixel color is computed by sorting overlapping Gaussians by depth and performing front-to-back alpha-compositing:

$$C = \sum_{i \in \mathcal{N}} \mathbf{c}_i \alpha_i G_i^{2D}(\mathbf{u}) \prod_{j=1}^{i-1} (1 - \alpha_j G_j^{2D}(\mathbf{u})) \quad (17)$$

where $G_i^{2D}(\mathbf{u})$ evaluates the projected 2D Gaussian at pixel $\mathbf{u}$, α_i is the learned opacity, and $\mathbf{c}_i = \sum_{l,m} k_{lm}^{(i)} Y_l^m(\mathbf{d})$ is the SH-evaluated color for viewing direction $\mathbf{d}$.

Key insight: Comparing Eq. (17) with Eq. (5), the image formation models are *mathematically identical*. The difference lies entirely in the rendering algorithm: NeRF samples points along rays (ray marching), while 3DGS projects primitives onto the image (rasterization).

4.5 Tile-Based Differentiable Rasterizer

The rasterizer proceeds as follows:

1. **Frustum culling:** Discard Gaussians outside the view frustum (99% confidence interval test).
2. **Tiling:** Divide the screen into 16×16 pixel tiles. Each Gaussian is assigned to all tiles it overlaps.
3. **Sorting:** Assign each (Gaussian, tile) pair a key encoding tile ID and depth. Sort all keys globally using a GPU radix sort.
4. **Forward pass:** For each tile, a CUDA thread block loads sorted Gaussians into shared memory and accumulates color via Eq. (17). Threads terminate when accumulated $\alpha \rightarrow 1$.
5. **Backward pass:** Traverse sorted lists back-to-front. Intermediate opacities are recovered by dividing the stored final accumulated opacity by each point's α , avoiding the need to store per-sample opacity lists.

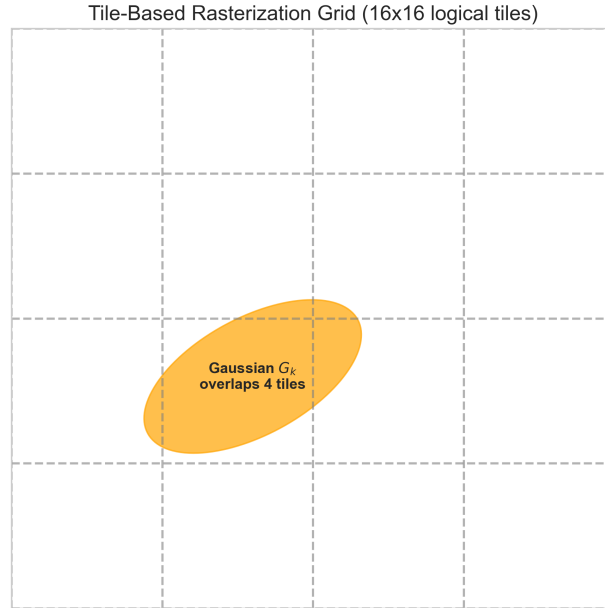


Figure 10: The tile-based differentiable CUDA rasterizer divides rendering processing bounds into 16×16 tiles to accelerate independent block atomic threads globally rendering overlapping opacities.

4.6 Optimization with Adaptive Density Control

4.6.1 Loss Function

The training loss combines an ℓ_1 photometric term with a structural similarity term:

$$\mathcal{L} = (1 - \lambda) \mathcal{L}_1 + \lambda \mathcal{L}_{\text{D-SSIM}}, \quad \lambda = 0.2 \quad (18)$$

4.6.2 Adaptive Gaussian Densification

Every 100 iterations, the optimizer modifies the Gaussian set:

ADAPTIVE DENSITY CONTROL IN COMPUTER VISION

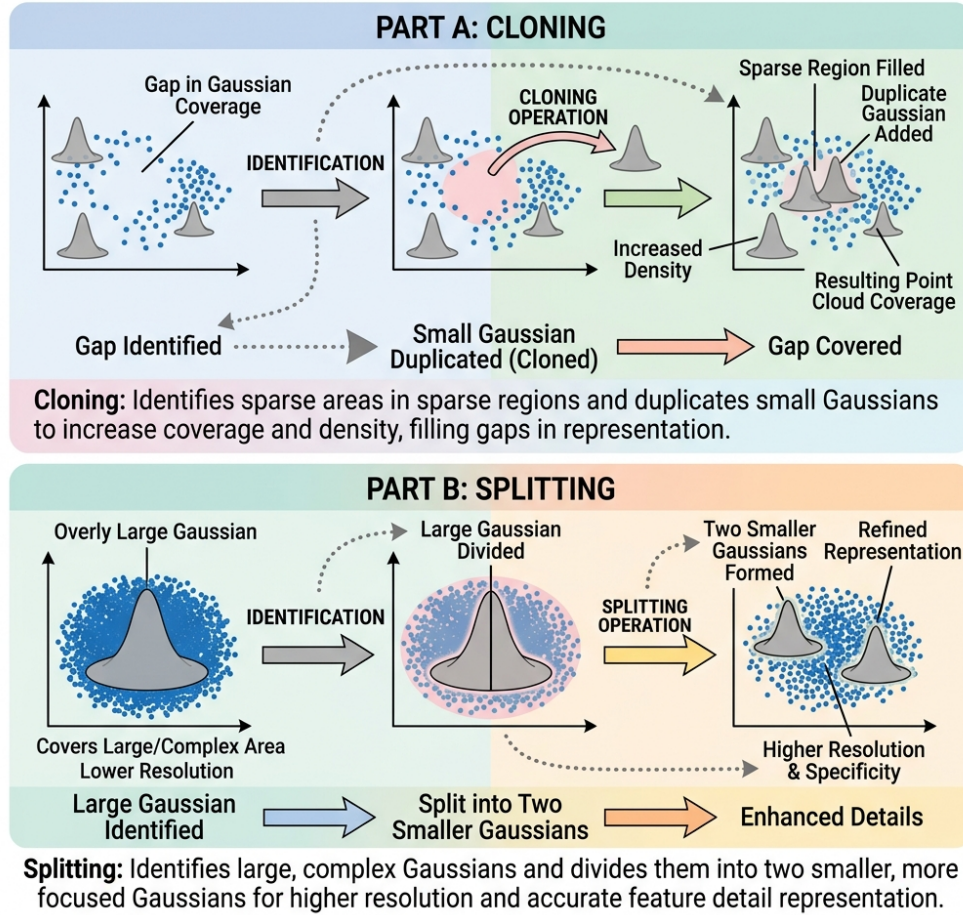


Figure 11: Adaptive density control: cloning is used for small-scale geometry in under-reconstructed areas, while splitting divides large Gaussians into smaller ones in regions with over-reconstruction gradients.

- **Densification criterion:** Gaussians with average view-space positional gradient magnitude $> \tau_{\text{pos}} = 0.0002$ are candidates.
- **Clone (under-reconstruction):** Small Gaussians in under-reconstructed regions are duplicated and displaced along the positional gradient direction.
- **Split (over-reconstruction):** Large Gaussians are replaced by two smaller ones (scale divided by $\phi = 1.6$), initialized by sampling from the parent Gaussian's distribution.
- **Pruning:** Gaussians with $\alpha < \epsilon_{\alpha}$ are removed. Periodically (every 3000 iterations), all opacities are reset near zero to allow culling of floaters.

4.7 3DGS Pipeline Summary

Algorithm 2 3D Gaussian Splatting Training

```

1: Initialize: Create Gaussians from SfM sparse point cloud
2: for iteration = 1, 2, ... do
3:   Select a training view; project all Gaussians to 2D (Eq. (16))
4:   Tile, sort, and rasterize via alpha-compositing (Eq. (17))
5:   Compute loss  $\mathcal{L}$  (Eq. (18)), backpropagate through rasterizer
6:   Update  $\{\boldsymbol{\mu}, \mathbf{q}, \mathbf{s}, \alpha, \mathbf{k}^{SH}\}$  via Adam
7:   if iteration mod 100 == 0 then
8:     Densify (clone/split) or prune Gaussians based on gradients
9:   end if
10: end for

```

5 Comparative Analysis: NeRF vs. 3D Gaussian Splatting

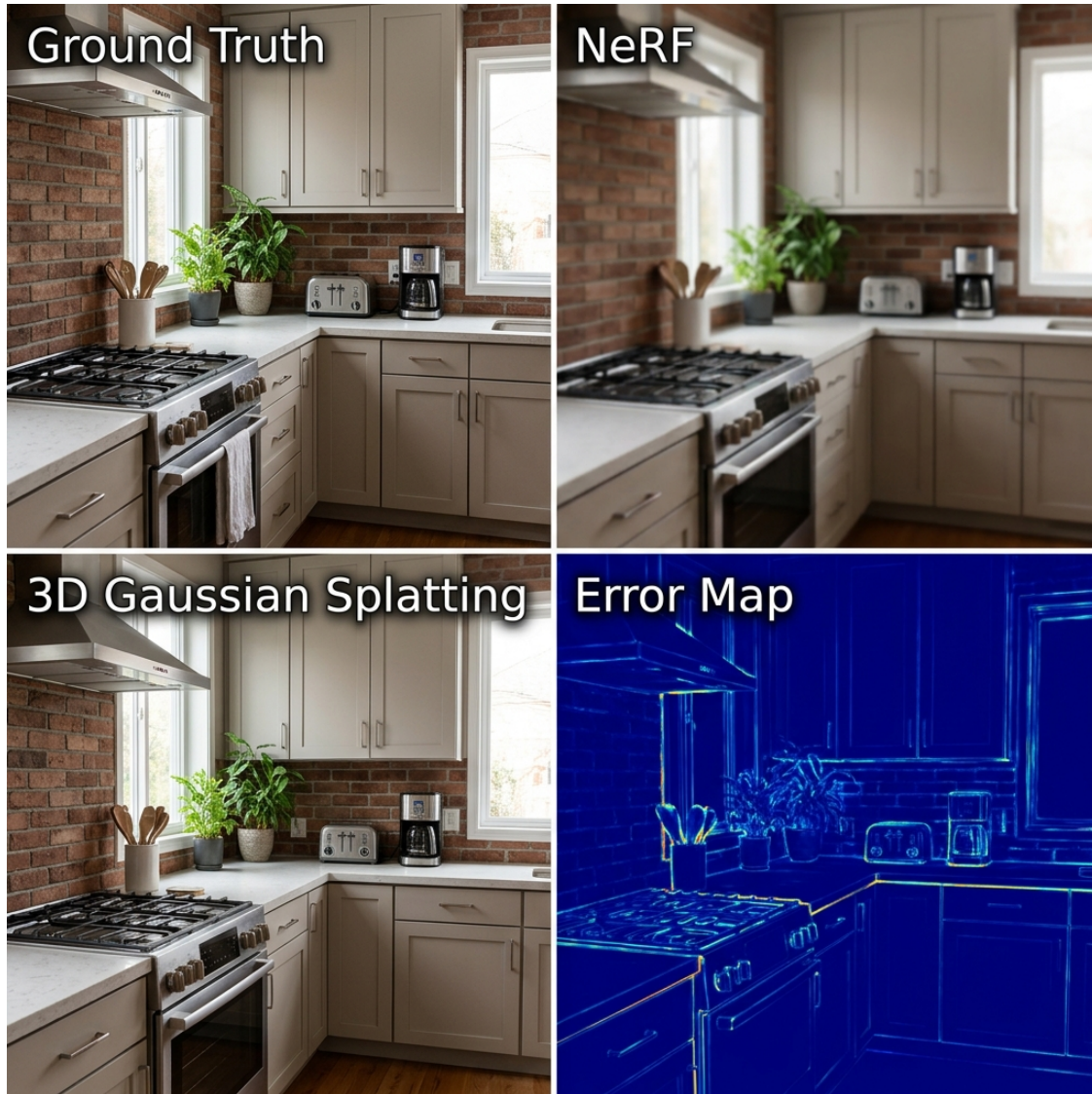


Figure 12: Typical visual comparison of novel view synthesis results. Top row: Ground Truth and NeRF; Bottom row: 3D Gaussian Splatting and the corresponding Error Map.

5.1 Representation

Table 1: Fundamental comparison of scene representation.

Property	NeRF	3DGS
Representation type	Implicit (MLP)	Explicit (point primitives)
Scene function	$F_{\Theta}(\mathbf{x}, \mathbf{d}) \rightarrow (\mathbf{c}, \sigma)$	$\{G_k : \boldsymbol{\mu}, \boldsymbol{\Sigma}, \alpha, SH\}_{k=1}^M$
Parameters	MLP weights ($\sim 1.2\text{M}$)	Per-Gaussian (~ 59 params $\times$ 1–5M)
Spatial structure	Unstructured (continuous)	Unstructured (discrete)
View-dependence	MLP output	Spherical harmonics
Topology changes	Implicit in weights	Explicit add/remove

5.2 Rendering Pipeline

Table 2: Rendering pipeline comparison.

Aspect	NeRF	3DGS
Rendering method	Ray marching	Tile-based rasterization
Samples per pixel	$N_c + N_f$ MLP queries	Variable (sorted splats)
Sorting	N/A (samples ordered on ray)	GPU radix sort
Empty space handling	Hierarchical sampling	Frustum culling + α pruning
Differentiability	Automatic (PyTorch)	Custom CUDA kernels

5.3 Quantitative Performance

Table 3: Performance comparison on Mip-NeRF360 dataset (from [2]).

Method	PSNR $\uparrow$	Training Time	Render FPS
Mip-NeRF360	24.3 dB	~ 48 hours	0.07
InstantNGP	22.1 dB	~ 7 min	9.2
Plenoxels	21.9 dB	~ 26 min	8.2
3DGS (short)	23.6 dB	~ 6 min	135
3DGS (full)	25.2 dB	~ 51 min	93

3DGS achieves *real-time* rendering (≥ 30 FPS at 1080p) while matching or exceeding the quality of Mip-NeRF360, with orders-of-magnitude faster training.

5.4 Geometric Analysis

NeRF’s **implicit representation** encodes scene geometry in the density field $\sigma(\mathbf{x})$. Surfaces can be extracted via level-set methods (marching cubes on σ), but the geometry is

entangled with the network weights and may contain floaters and noise.

3DGS’s explicit representation directly provides geometric primitives. The Gaussian means μ_k approximate the scene surface, and the covariance matrices Σ_k encode local surface orientation (the smallest eigenvector approximates the surface normal). However, 3DGS does not enforce surface constraints, so Gaussians can be semi-transparent volumetric blobs rather than precise surface elements.

5.5 Implicit Smoothing vs. Explicit Detail

The observed visual differences in rendering quality—particularly the tendency for NeRF to produce slightly “blurry” or “foggy” results compared to the sharp reconstructions of 3DGS—can be attributed to three core mathematical and architectural factors:

1. **Spectral Bias of MLPs:** As discussed in Section 3.2, coordinate-based neural networks are inherently biased toward learning low-frequency functions. Even with positional encoding ($L = 10$), the MLP effectively acts as a continuous interpolator over the 5D radiance field. This creates a natural “low-pass filtering” effect, where sharp photometric transitions (e.g., fine textures, thin edges) are smoothed over by the network’s internal activation manifolds.
2. **Volumetric Dispersion vs. Delta Localization:** In NeRF’s volume rendering (Eq. 5), the color of a pixel is an integral of radiance weighted by density. If the optimization does not perfectly localize the density σ to a Dirac delta-like distribution at the exact surface boundary, the density becomes “smeared” across a small depth interval. This volumetric dispersion results in a semi-transparent blur. Conversely, 3DGS uses anisotropic Gaussians that can be optimized to have extremely small scales along the surface normal direction, allowing for highly localized “splating” that preserves sharp high-frequency details.
3. **Fixed Parameter Budget vs. Adaptive Densification:** NeRF’s capacity is fundamentally capped by the fixed number of weights in its MLP ($\sim 1.2\text{M}$ parameters for the standard model). 3DGS, however, utilizes an adaptive densification scheme (Section 4.6) that grows from a sparse input to millions of primitives. This allows the model to “allocate” additional capacity (more Gaussians) specifically where high-frequency geometric detail persists, effectively increasing local resolution beyond what is feasible for a globally-optimized set of neural weights.

6 Connection to Course Topics in GMCV

6.1 Projective Geometry and Camera Models

Both methods require calibrated cameras. NeRF uses $(\mathbf{K}, \mathbf{R}, \mathbf{t})$ to cast rays (Eq. (2)), while 3DGS uses them for Gaussian projection. The intrinsic matrix entries directly affect the Jacobian $\mathbf{J}$ in EWA splatting (Eq. (16)).

6.2 Epipolar Geometry and Multi-View Consistency

NeRF's density-only-from-position constraint implicitly enforces multi-view consistency: the same 3D point must have the same density regardless of viewing direction. 3DGS achieves this through explicit 3D positioning of Gaussians visible across views.

6.3 Structure from Motion (SfM)

3DGS directly uses the SfM sparse point cloud as initialization. NeRF requires only camera poses from SfM. Both pipelines depend critically on accurate camera calibration.

6.4 Optimization on Manifolds

The quaternion parameterization $\mathbf{q} \in \mathbb{S}^3$ in 3DGS is an optimization on a manifold, connecting to Lie group theory on $SO(3)$ covered in the course. The covariance decomposition $\Sigma = \mathbf{R}\mathbf{S}\mathbf{S}^T\mathbf{R}^T$ is the Cholesky-like parameterization ensuring the positive-definite cone constraint.

7 Illustrative Code Implementations

7.1 NeRF: Positional Encoding and Volume Rendering

```

1 import torch
2 import torch.nn as nn
3 import numpy as np
4
5 class PositionalEncoding(nn.Module):
6     """Maps input coordinates to higher-dimensional space using
7     sinusoidal functions to overcome spectral bias of MLPs."""
8     def __init__(self, L=10):
9         super().__init__()
10        self.L = L
11        # Frequency bands: 2^0, 2^1, ..., 2^{L-1}
12        self.freq_bands = 2.0 ** torch.linspace(0, L-1, L)
13
14    def forward(self, x):
15        # x: (... , D) -> (... , D * 2L)
16        encodings = []
17        for freq in self.freq_bands:
18            encodings.append(torch.sin(freq * np.pi * x))
19            encodings.append(torch.cos(freq * np.pi * x))
20        return torch.cat(encodings, dim=-1)
21
22 def volume_render(sigmals, colors, deltas):
23     """Discrete volume rendering (Eq. 5 in report).
24     Args:
25         sigmas: (N,) volume densities
26         colors: (N, 3) RGB colors
27         deltas: (N,) distances between adjacent samples
28     Returns:

```

```

29     rendered_color: (3,) accumulated RGB
30     """
31     # alpha_i = 1 - exp(-sigma_i * delta_i)
32     alphas = 1.0 - torch.exp(-sigmas * deltas)
33     # T_i = prod_{j<i} (1 - alpha_j) [exclusive cumprod]
34     T = torch.cumprod(
35         torch.cat([torch.ones(1), 1.0 - alphas[:-1]]), dim=0
36     )
37     # C = sum_i T_i * alpha_i * c_i
38     weights = T * alphas
39     rendered_color = (weights.unsqueeze(-1) * colors).sum(dim=0)
40     return rendered_color

```

Listing 1: NeRF positional encoding and volume rendering.

7.2 3DGS: Gaussian Projection and Covariance Computation

```

1  import torch
2  import torch.nn.functional as F
3
4  def quaternion_to_rotation(q):
5      """Convert unit quaternion (w,x,y,z) to 3x3 rotation matrix.
6      Implements Rodrigues-like formula on SO(3)."""
7      q = F.normalize(q, dim=-1)
8      w, x, y, z = q.unbind(-1)
9      R = torch.stack([
10         1 - 2*(y*y + z*z), 2*(x*y - w*z), 2*(x*z + w*y),
11         2*(x*y + w*z), 1 - 2*(x*x + z*z), 2*(y*z - w*x),
12         2*(x*z - w*y), 2*(y*z + w*x), 1 - 2*(x*x + y*y)
13     ], dim=-1).reshape(-1, 3, 3)
14     return R
15
16  def build_covariance(scales, quats):
17      """Build 3D covariance: Sigma = R @ S @ S^T @ R^T
18      Guarantees positive semi-definiteness by construction."""
19      R = quaternion_to_rotation(quats) # (M, 3, 3)
20      S = torch.diag_embed(scales) # (M, 3, 3)
21      RS = R @ S # (M, 3, 3)
22      Sigma = RS @ RS.transpose(-1, -2) # (M, 3, 3)
23      return Sigma
24
25  def project_to_2d(Sigma_3d, W, J):
26      """EWA splatting: project 3D covariance to 2D.
27      Sigma_2d = J @ W @ Sigma_3d @ W^T @ J^T
28      Then extract top-left 2x2 block."""
29      JW = J @ W # (M, 3, 3)
30      Sigma_cam = JW @ Sigma_3d @ JW.transpose(-1, -2)
31      Sigma_2d = Sigma_cam[:, :2, :2] # (M, 2, 2)
32      return Sigma_2d

```

Listing 2: 3DGS covariance construction and 2D projection.

8 Discussion and Conclusion

8.1 Summary of Findings

NeRF and 3DGS represent two complementary paradigms for novel view synthesis:

- **NeRF** pioneered the use of coordinate-based neural representations with differentiable volume rendering, achieving photorealistic quality but at the cost of slow training ($\sim 1\text{--}2$ days) and rendering (~ 30 seconds per frame).
- **3DGS** demonstrated that explicit, differentiable primitives with a custom GPU rasterizer can match or exceed NeRF quality while achieving real-time rendering (>100 FPS) and fast training ($\sim$ minutes).

8.2 Limitations and Open Problems

- **NeRF:** Computational cost, difficulty with dynamic scenes, reliance on dense views, limited to bounded scenes without extensions (Mip-NeRF 360).
- **3DGS:** High memory consumption (storing millions of Gaussians), no topological guarantees, difficulty extracting clean meshes, popping artifacts from approximate sorting.
- **Both:** Rely on accurate camera calibration, struggle with transient objects, limited generalization (per-scene optimization).

8.3 Future Directions

Active research directions include: generalizable feed-forward models (pixelNeRF, pixelSplat), dynamic scene extensions (D-NeRF, 4D Gaussians), compression of Gaussian representations, hybrid implicit-explicit methods, and integration with SLAM and robotics pipelines.

References

- [1] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “NeRF: Representing scenes as neural radiance fields for view synthesis,” in *ECCV*, 2020.
- [2] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3D Gaussian splatting for real-time radiance field rendering,” *ACM Trans. Graph.*, vol. 42, no. 4, 2023.
- [3] M. Zwicker, H. Pfister, J. van Baar, and M. Gross, “EWA volume splatting,” in *IEEE Visualization*, 2001.
- [4] J. T. Barron, B. Mildenhall, D. Verbin, P. P. Srinivasan, and P. Hedman, “Mip-NeRF 360: Unbounded anti-aliased neural radiance fields,” in *CVPR*, 2022.
- [5] T. Müller, A. Evans, C. Schied, and A. Keller, “Instant neural graphics primitives with a multiresolution hash encoding,” *ACM Trans. Graph.*, vol. 41, no. 4, 2022.
- [6] S. Fridovich-Keil, A. Yu, M. Tancik, Q. Chen, B. Recht, and A. Kanazawa, “Plenoxels: Radiance fields without neural networks,” in *CVPR*, 2022.
- [7] N. Rahaman et al., “On the spectral bias of neural networks,” in *ICML*, 2019.
- [8] M. Tancik et al., “Fourier features let networks learn high frequency functions in low dimensional domains,” in *NeurIPS*, 2020.
- [9] J. L. Schönberger and J.-M. Frahm, “Structure-from-motion revisited,” in *CVPR*, 2016.
- [10] N. Max, “Optical models for direct volume rendering,” *IEEE TVCG*, vol. 1, no. 2, 1995.